GlobalStack: Local Azure Emulation with Layered Outcome Checks

Technical draft. Author information pending.
Native snapshot: 2026-11-04; local fault study: 2026-11-04 UTC; native claims case: 2026-11-06 UTC

Abstract

Infrastructure creation and workflow completion are incomplete checks of an application's intended behavior. GlobalStack provides selected local Azure management and service interfaces through which native clients and HTTP workflows can be exercised. We present an executable, scoped validation method and a controlled comparison of five outcome checks. In two first-party scenarios, we apply 12 explicit configuration mutations and run connected, mutated, and restored episodes in 3 fresh-state repetitions, totaling 108 episodes. Management acceptance detects 0/12 distinct mutations; workflow status detects 2/12; intended arrival detects 5/12; exact payload and cardinality at the intended destination detect 10/12; and excluding additional observed deliveries detects 12/12. All 72 positive episodes pass every check. The difference between the last two checks exposes a limitation in our earlier outcome predicate: a correct result can coexist with delivery to an unintended destination. These are sensitivity results for deliberately constructed local faults, not estimates of real-world fault prevalence or Azure fidelity. A separate native matrix validates one sample in each of 9 platform configurations. A separate native Terraform case adds a Blob, Event Grid, Queue and Table path, with 21 episodes across 3 fresh runtimes. It detects three selected wiring faults and exposes a duplicate-insert defect, subsequently corrected and rerun. We retain the protocol, traces, source identifiers, cleanup results, and explicit limitations. No live Azure comparison or independently authored workload evaluation supports the cross-service result.

1 Problem and scope

Infrastructure-as-code (IaC) tools describe resources and their relationships, but successful creation is only one requirement of an application. A workflow can reference an existing but incorrect secret. A publisher can accept an event after its subscription is deleted. Both conditions can leave resources readable and operations apparently successful while the business outcome is absent or incorrect.

GlobalStack is a local development system for exercising selected Azure deployments and service interactions. Its public preview is intended for trusted workloads on isolated development hosts. The contribution described here is an executable validation method and its implementation: native provisioning, observable HTTP service interactions, deliberate wiring faults, restoration, and cleanup. This paper does not introduce a new model benchmark, report model-generated deployments, or claim complete Azure emulation.

1.1 Research questions and contributions

RQ1: Native execution. Can the selected native clients provision, inspect, and remove their reference samples? The unit of evidence is one configuration per platform, rather than a platform-wide compatibility claim.

RQ2: Checker sensitivity. On the same deliberately faulty configurations, which failures are detected by management acceptance, workflow status, arrival, exact-at-target, and exclusive-outcome checks? Comparing checks on identical traces holds the executed configuration constant.

RQ3: Observation boundaries. Does correct delivery at the intended target exclude unwanted delivery elsewhere? The experiment includes duplicate and additional-destination mutations to test this distinction directly.

RQ4: Local repeatability and restoration. Do all check vectors repeat across fresh-state runs, do restored configurations regain the intended outcome, and do the addressed resources disappear after cleanup?

The contributions are a scoped implementation connecting management state to executable service interactions; an auditable configuration-fault experiment with explicit controls and denominators; and a documented strengthening of an outcome predicate that missed additional destinations. Fault injection, end-to-end assertions, and differential testing are established techniques. We claim an application and measured engineering result, not invention of those techniques or a general proof of Azure compatibility.

2 System design

2.1 Management and service state

The management server exposes supported resource and deployment routes over HTTP. A resource store represents management records. Resource creation and deletion invoke runtime synchronization hooks, which connect those records to service-specific state. Selected data-plane routers expose executable operations, including Key Vault secret reads and Event Grid publication and webhook delivery. A local workflow runner evaluates supported expressions and performs supported actions.

This separation matters for correctness. A management record can be correct while a runtime endpoint still serves stale state. Conversely, a valid deployment can connect the wrong runtime resources. The validation therefore reads management resources, exercises service endpoints, and verifies cleanup across both surfaces.

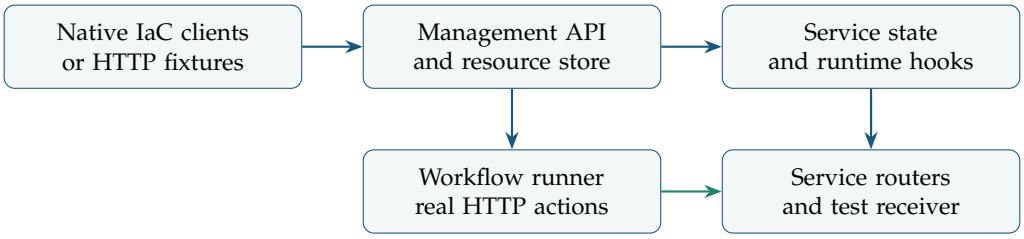


Figure 1: The evaluated local architecture. The test receiver is a small HTTP application, not an emulated Azure Functions deployment. Service support is operation-specific.

2.2 Native runners

The native matrix includes Terraform and OpenTofu using AzAPI, Pulumi TypeScript, Python and YAML using Azure Native, Ansible, Azure CLI REST, Bicep, and ARM JSON. Generated runner contexts separate tool execution from endpoint routing. The harness selects the local provider and supplies a development identity. This identity supports the local protocol path; it does not reproduce Azure tenant isolation or RBAC enforcement.

Each sample runs build, lint, unit tests, deploy, provider lifecycle, integration tests, and destroy. The meaning of a stage depends on the platform. In particular, the integration stage in the native reference samples does not by itself establish an application event path. The separate cross-service experiments below supply that evidence. Terraform results concern AzAPI, not the AzureRM provider.

2.3 Support boundaries

Registered resource types, accepted management operations, executable service behavior, and live-cloud conformance are distinct levels of evidence. GlobalStack's service scope and conformance documents distinguish these levels. A registration count cannot measure fidelity. The broader historical 30-service validator still depends on missing artifacts and optional backend matrices and is not reported as passing in this snapshot.

The tested state is ephemeral. Unsupported paths and invalid inputs have selected rejection tests, but this paper makes no universal fail-closed claim. Workflow expression coverage is partial. Retry policy,

delivery ordering, eventual delivery, sustained throughput, durable recovery, and multi-tenant isolation are not established by these experiments.

3 Validation method

3.1 Fixtures and provenance

The earlier snapshot uses three fixture groups. The native group contains one first-party sample per platform. These are smoke configurations, not nine independent application architectures. The upstream group contains three unmodified Microsoft Azure Quickstart templates: Storage account creation, Key Vault creation, and Event Grid [1]. Their original bytes, source revision, SHA-256 digests, and MIT license are retained. The Storage example checks management behavior. Key Vault adds a secret read over HTTP. Event Grid adds publication and exact webhook checks with broken-routing and restoration controls.

The third group contains two new first-party scenarios. Their inputs and outcome checks are maintained in the same repository as the emulator. This enables regression testing but creates an oracle-bias risk. Neither first-party fixtures nor unmodified upstream templates establish live Azure equivalence on their own. The configuration-fault study reuses the two first-party architectures; its additional episodes must not be counted as additional architectures.

3.2 Two cross-service scenarios

Configuration enrichment. A Logic Apps workflow reads a configuration value from Key Vault and posts an enriched course record to an HTTP receiver. The intended result contains the request identifier, course identifier, and the expected office value. One control changes the workflow to read an existing secret containing the wrong office. A second control references a missing secret. Restoring the original reference must restore the exact result.

Event routing. A Logic Apps workflow transforms a course record into an Event Grid event and publishes it to a topic. A scoped subscription delivers the event to the intended receiver. One control deletes the subscription. A second recreates it with the wrong destination. Restoring the intended destination must restore delivery. The input includes a unique request identifier and timestamp, and the expected event includes the exact identifier, subject, event type, timestamp, data version, and data object.

Both scenarios create resources using the management API and check successful deployment and resource read-back. The workflow uses its default HTTP transport. Test-owned loopback ports map incoming requests to fixed Azure hostnames at the router boundary; no service response or workflow transport is mocked. This routing adaptation is part of the local harness and is not evidence about Azure DNS, TLS, or authentication behavior.

3.3 Outcome and control predicates

For an invocation i, let r_i be its unique identifier, d_i its intended receiver path, and p_i its expected JSON payload. Let R_i be the sequence of deliveries observed during that invocation with the same identifier. Retain repeated deliveries as separate observations and define the filtered sequence

$$A_i = [x \in R_i : \text{path}(x) = d_i], \tag{1}$$

$$O_i = (|A_i| = 1) \ \wedge \ (\text{body}(A_i[1]) = p_i). \tag{2}$$

The second term is evaluated only when the first is true. JSON equality here is structural and type-sensitive. An event at the wrong path, a missing event, a duplicate at the intended path, or a wrong field fails O_i. Exactly one delivery means one observed delivery in the local test interval; it is not an exactly-once delivery guarantee for a distributed service.

We separately record deployment acceptance D_i, observed service-boundary HTTP traffic H_i, and workflow success W_i. A positive episode requires $D_i \wedge H_i \wedge W_i \wedge O_i$. A deliberately broken episode requires $D_i \wedge H_i \wedge \neg O_i$ plus its specific expected status or destination observation. Thus a control passes the test when it demonstrates the intended failure. The harness does not relabel a broken application as a working deployment.

For each scenario, cleanup C requires successful group deletion, management reads returning 404 for the group and workflow, and a request to the deleted service's data-plane endpoint returning 404. A complete scenario must satisfy the four episode expectations and C. This checks the addressed resources, not the absence of every possible process, cache, or background task.

3.4 Configuration-fault study

The new experiment extends the same two scenarios with six mutations each. Table 4 lists every case. The fault classes follow the observable obligations of these flows: correct references, correct routing, preserved payloads, the required number of deliveries, and accepted service input types. This selection deliberately probes those obligations; it is not sampled from an incident corpus. Each mutation changes one field or relationship, or adds one delivery relationship. The expected business payload remains unchanged.

For every case, a new resource store, runtime, and set of loopback listeners execute connected, mutated, and restored episodes, followed by cleanup. All 12 cases run sequentially in the declared order in three repetitions. This produces 36 case executions and 108 episodes, including 36 mutated and 72 positive episodes. Repetition tests local consistency; it does not increase the number of distinct mutations or independent applications. The protocol and executable cases were committed before the first study run, but were not externally preregistered. Review then separated prior trigger responses from the next setup observation; the corrected run preserves the same per-episode decisions. Both complete records are retained and reported separately, not pooled.

Let B_i contain *all* deliveries to the instrumented receiver during invocation i, including wrong-path or wrong-identifier deliveries. The interval starts immediately before the trigger request and ends when its response body has been consumed. This is meaningful for the evaluated synchronous runtime, which awaits downstream requests. It is not a sufficient observation window for asynchronous cloud delivery. The earlier R_i and A_i are identifier- and destination-filtered subsequences of B_i.

Let M_i mean that recorded setup or mutation operations were accepted, the deployment reports success, and the workflow is readable. W_i requires trigger acceptance and successful workflow completion. These are intentionally shallow baselines: M_i does not inspect every semantic resource relationship. The five nested checks are

$$K_i^M = M_i, \qquad K_i^W = M_i \wedge W_i, \tag{3}$$

$$K_i^A = K_i^W \wedge (|A_i| \geq 1), \qquad K_i^T = K_i^W \wedge O_i, \tag{4}$$

$$K_i^E = K_i^T \wedge (|B_i| = 1). \tag{5}$$

The last check adds absence of extra observed delivery to the exact-at-target predicate. It applies to the specified single-destination contract. Legitimate fan-out applications require a different expected destination set and cardinality.

For the enumerated mutated episodes F and positive episodes P, report the rejection fractions

$$\text{detect}(k) = \frac{\sum_{i \in F} \mathbf{1}[\neg K_i^k]}{|F|}, \qquad \text{positiveReject}(k) = \frac{\sum_{i \in P} \mathbf{1}[\neg K_i^k]}{|P|}. \tag{6}$$

These fractions describe the declared experiment. They are not estimates of population sensitivity or specificity. Mutants rejected by service validation remain in F. Harness errors are recorded as inconclusive, rather than checker detections; none occurred in the retained study. A checkpoint is written before each attempt and updated after completion. A missing or incomplete attempt prevents accepting a complete study result. No mutations were removed as equivalent or unexercised.

The artifact retains management status observations, service-boundary traffic, expected payloads, all observed receiver deliveries, and cleanup outcomes. Checker decisions are recomputed from these traces. Generated tables also verify each mutation's concrete witness, such as a wrong course value, a second receiver request, or a service rejection. No measured performance or cost comparison is inferred from this experiment.

3.5 Refining a native corpus task

A separate audit of a private verification corpus found 50 bundles with 35 scenario labels: 30 Pulumi TypeScript, five Pulumi Python, five Terraform, five OpenTofu and five Bicep. All 50 source-unit commands

passed on rerun. The 50 outer historical reports record successful seven-stage execution, but the integration sources contain only two distinct implementations and check management resources and outputs. They do not establish business-event propagation. Labels and resource multisets are not a validated architecture count. These historical reports are not new execution results.

We selected the Terraform claims-evidence task for an explicitly versioned reference refinement. This is a purposive first case, not a random sample, unchanged model output or externally selected workload. The original prompt and reports remain intact. The revised contract adds an application producer that uploads synthetic evidence and explicitly publishes a correlated Event Grid event. Terraform creates the topic-scoped subscription. A reference HTTP worker reads the blob, computes its byte digest, writes a classification message to a Storage Queue and writes a ledger entity to a Storage Table. This path does not use the workflow engine. The worker is test application code, not an emulated Function App, and blob upload does not automatically generate the event.

For episode i, let D_i be all callbacks at the instrumented receiver in the synchronous publish interval, Q_i the observed queue records with the episode identifier, e_i the expected event and q_i the expected record. Let L_i mean that the addressed ledger row exists with the expected partition key, row key, identifier, claim, blob name, byte digest and status. The predicate is

$$C_i = (|D_i| = 1) \wedge (\text{path}(D_i[1]) = \texttt{/claims}) \wedge (\text{body}(D_i[1]) = [e_i]) \wedge (Q_i = [q_i]) \wedge L_i. \tag{7}$$

As above, indexing requires a satisfied cardinality guard. Queue observation peeks at most 32 messages; each isolated run remains below this bound. Ledger comparison allows service metadata such as timestamps and ETags. It does not audit every row or uninstrumented side effect.

Each fresh run executes connected, wrong-destination, restored, missing-subscription, restored, duplicate-subscription and restored episodes. Native Terraform applies every change and must produce a no-change plan after each apply. Fault-specific witnesses distinguish an absent route, a wrong callback path and two callbacks with two queue records from incidental harness failures. The bundle's integration entry point requires the behavioral context and cannot fall back to its former management-only check. Destruction and addressed-resource disappearance are checked separately.

3.6 Execution and reproducibility

The snapshot uses Linux x86_64, four logical CPUs, and approximately 31 GiB of RAM. Native jobs run in three isolated checkout groups. A disk guard stops work before the required 40 GiB free-space reserve is crossed. These are observed host characteristics, not minimum hardware requirements. No live Azure deployment or paid model request is part of this validation.

Pinned versions cover the principal CLI tools and direct sample SDK dependencies. The TypeScript sample uses an npm lockfile and `npm ci`; the root workspace also has a lockfile. The native run uses one standalone Terraform base and four derived validation images to share layers. Their recipes and observed image identifiers are retained. We have not built every standalone runner Dockerfile in this experiment. Mutable base tags, operating-system packages, upstream installation scripts, and unpinned Python transitive dependencies remain. The artifact supports rerunning the declared checks, not a byte-identical image rebuild claim.

4 Results

4.1 Native and local checks

Table 1 reports the current native matrix. Each row is one sample and all seven stages are required, including destruction. The provider-lifecycle reports contain 50 operation checks across the nine runs. These include overlapping checks and are not a count of distinct Azure APIs.

In the earlier native/flow snapshot, the local conformance command reports 559 passing tests and 1 explicitly opt-in Docker-dependent Kubernetes test skipped. Repository policy tests report 19 passes. Three upstream template tests are included in the local suite. Separately executed Key Vault, Event Grid, and Logic Apps packs report 99, 45, and 62 cases respectively. These suites overlap and must not be added to form an Azure-coverage score. The archived preview-2 result is an earlier snapshot with 556 local passes; this draft's record identifies the new runtime and evidence revisions explicitly.

Table 1: Native smoke matrix. Each row has one sample.

Configuration	Native tool or SDK	Result
Ansible	Core 2.21.6; collection 3.21.0	7/7; cleanup passed
ARM JSON	Azure CLI 2.92.0	7/7; cleanup passed
Bicep	Bicep 0.48.1; Azure CLI 2.92.0	7/7; cleanup passed
Azure CLI REST	Azure CLI 2.92.0	7/7; cleanup passed
OpenTofu HCL	OpenTofu 1.13.2; AzAPI 2.13.0	7/7; cleanup passed
Pulumi Python	CLI 3.273.0; Azure Native 3.30.0	7/7; cleanup passed
Pulumi TypeScript	CLI 3.273.0; Azure Native 2.92.3	7/7; cleanup passed
Pulumi YAML	CLI 3.273.0; Azure Native 3.18.0	7/7; cleanup passed
Terraform HCL	Terraform 1.17.1; AzAPI 2.13.0	7/7; cleanup passed

4.2 Cross-service observations

Table 2 retains the original eight-episode experiment, separate from the later configuration-fault study. Both connected and restored episodes in each scenario produce the exact expected payload. All four deliberately broken configurations fail the application outcome assertion as intended. Three of these four still report workflow success. This demonstrates a specific limitation of using operation success as the only local wiring check. It is not a measured prevalence of such failures in real deployments.

Table 2: Constructed controls and positive episodes. "No" in the outcome column is expected for the broken configurations.

Scenario	Configuration	Workflow	Exact outcome
Enrichment	connected	Succeeded	Yes
Enrichment	wrong-config-reference	Succeeded	No (expected)
Enrichment	missing-config-reference	Failed	No (expected)
Enrichment	restored	Succeeded	Yes
Event routing	connected	Succeeded	Yes
Event routing	deleted-subscription	Succeeded	No (expected)
Event routing	wrong-destination	Succeeded	No (expected)
Event routing	restored	Succeeded	Yes

Both scenario cleanup checks pass. The evidence records the unique input, service-boundary status, receiver path and payload, workflow status, assertion result, and cleanup result. The test report is generated only after all assertions succeed. Failed test processes must be retained separately; absence of a report is not a passing result.

4.3 Checker ablation and repeatability

Table 3 answers RQ2. Management acceptance rejected none of the 12 distinct mutations. This result reflects the chosen semantic faults and the shallow baseline; it does not imply that management validation never detects infrastructure errors. Workflow status rejected the missing-secret and invalid-event-schema cases. Arrival additionally rejected missing and wrong routes. Exact-at-target additionally rejected wrong values and duplicate deliveries at the intended target. Exclusive outcome additionally rejected the two extra-destination cases.

Table 3: Rejected faulty episodes and rejected positive controls. Distinct faults count the 12 configurations once; repetitions do not add independent faults.

Check	Distinct faults	Repeated faults	Positive rejected
Management acceptance	0/12	0/36	0/72
Workflow success	2/12	6/36	0/72
Intended arrival	5/12	15/36	0/72
Exact at target	10/12	30/36	0/72
Exclusive outcome	12/12	36/36	0/72

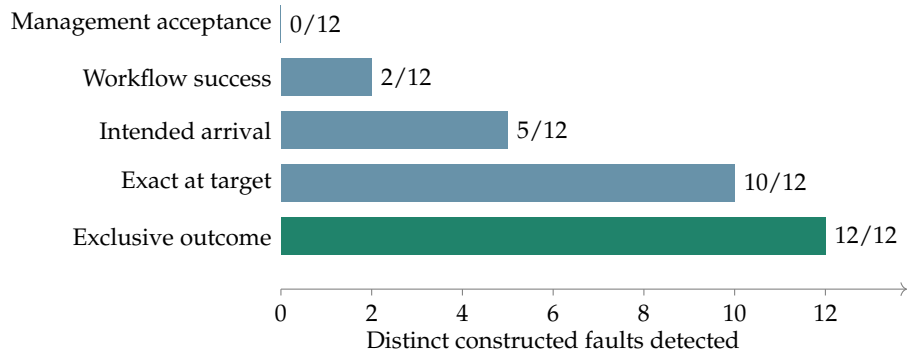


Figure 2: Adding observations exposes different faulty configurations. The final bar is coverage of the selected mutations, not an Azure compatibility score.

RQ3 has a concrete counterexample. In each extra-destination case, one exact expected delivery arrives at `/accepted` and another arrives at `/wrong`. The workflow succeeds and O_i remains true because A_i excludes the wrong path. The stricter predicate rejects the additional observed delivery. This finding concerns the test oracle, not a newly discovered emulator defect: the emulator executes the extra configured relationship. Tests concerned with unintended side effects must state and observe that obligation explicitly.

For RQ4, all five check decisions were identical across the three repetitions of every case. All 72 connected or restored episodes passed all checks, and all 36 case executions passed their cleanup assertions. There were zero inconclusive attempts. These observations establish repeatability on one local host and successful restoration for these inputs. They do not establish stability across hosts, runtime versions, delayed delivery, or failures outside the instrumented boundary.

Table 4: Per-mutation detections, identical in all three repetitions. X means the check rejected the faulty configuration; a dot means it accepted it. M: management; W: workflow; A: arrival; T: exact at target; E: exclusive outcome.

Flow	Mutation	Class	M	W	A	T	E
Vault	wrong-config-reference	reference	.	.	.	X	X
Vault	missing-config-reference	reference	.	X	X	X	X
Vault	wrong-destination	routing	.	.	X	X	X
Vault	wrong-course	payload	.	.	.	X	X
Vault	duplicate-delivery	multiplicity	.	.	.	X	X
Vault	extra-destination	multiplicity	.	.	.	.	X
Events	deleted-subscription	routing	.	.	X	X	X
Events	wrong-destination	routing	.	.	X	X	X
Events	wrong-course	payload	.	.	.	X	X
Events	duplicate-delivery	multiplicity	.	.	.	X	X
Events	extra-destination	multiplicity	.	.	.	.	X
Events	wrong-version-type	schema	.	X	X	X	X

4.4 Native claims case and repair

Table 5 reports the corrected native case. All 12 positive episodes satisfy C_i, and all 9 faulty episodes are rejected with their specified witnesses. Every native apply and no-change plan succeeds, including those for deliberately wrong wiring. The three repetitions run separate runtimes concurrently on one host; they are not independent replications. Each destruction succeeds, and the resource group, 11 listed management resources and four addressed data-plane surfaces return 404. The minimum observed free disk is 107.15 GiB. Terraform is 1.17.1 and AzAPI is 2.13.0.

Review of the earlier three clean-revision runs exposed a storage defect: the duplicate-subscription case inserted the same Table entity twice and received two successful responses. The documented Azure error contract specifies an existing-entity conflict [11]. The router now rejects the duplicate with HTTP 409 and preserves the original entity, including its metadata. A regression covers identical and changed duplicate values, unchanged read-back, and distinct keys. The reference worker checks an existing row after that conflict; it still writes two queue messages in this deliberate duplicate-subscription fault, so the outcome predicate rejects it. The corrected runs retain the 201, then 409 insert responses. Pre-fix observations remain archived separately and are not pooled into a larger final sample. This is a documentation-grounded correction with local regression evidence, not a live-cloud measurement.

Table 5: One refined native scenario. Callback and message counts are per episode. Rejection is the expected outcome for the three fault configurations.

Configuration	Repeats	Callbacks	Messages	Outcome
Connected / restored	12	1	1	Pass
Wrong destination	3	1 (wrong)	0	Reject
No subscription	3	0	0	Reject
Two subscriptions	3	2	2	Reject

The revised path broadens the evaluated implementation beyond the workflow engine, but remains first-party. Service Bus, Event Hubs, Files and App Configuration are also provisioned by the bundle but are outside the executed event path. Real Azure permits Event Grid redelivery and requires webhook endpoint validation [9, 10]; this receiver does not implement a production deduplication policy or that handshake. A paired live comparison must first implement those obligations and replace the synchronous window with a declared asynchronous observation policy. Passing this local case cannot establish those cloud properties.

4.5 Defects found through executable checks

Literal-value coercion. The workflow evaluator previously converted numeric-looking JSON strings into numbers outside expression argument parsing. An Event Grid event with the literal data version `"1.0"` reached the service as a number and failed its schema check. The correction preserves literal JSON types while parsing numeric constants inside supported expression arguments. A regression test also covers leading-zero strings, quoted literal text, numbers, booleans, arrays, and existing supported arithmetic. The Azure expression reference distinguishes expressions from literal values [2]; this fix is not a claim to implement the full expression language.

Management deletion without runtime deletion. Resource-group deletion previously removed management records without invoking deletion hooks for the resources in that group. A Key Vault secret remained readable after its group disappeared. The correction enumerates the group's resources, invokes the runtime deletion hooks in child-first order, and is checked by the new post-delete data-plane assertions. The selected regression establishes cleanup for these flows; other runtime backends require their own checks.

These findings illustrate why read-back, live requests, and cleanup are separate obligations. The repaired code is evaluated on the same authored scenarios, so these findings should be interpreted as engineering regressions rather than an independent comparative study.

5 Relation to software-testing research

A test oracle distinguishes expected from incorrect behavior. Barr et al. survey specification-based, derived, and other oracle approaches [6]. Our expected business payload and delivery contract are manually specified oracles. This explains both their value and their limitation: the original predicate omitted an obligation that its successful checks could not establish. More successful executions of that predicate would not remove the omission.

Mutation testing evaluates tests using deliberately introduced faults. Petrović et al. study how mutation testing affects testing practice and its relationship to real faults [7]. Our experiment instead mutates selected infrastructure relationships and payload fields while keeping the emulator implementation fixed. It compares observation layers using the same traces. We do not establish that our mutations represent a real incident distribution or inherit empirical validity from a different mutation study.

Differential testing compares behavior across implementations to expose disagreements [8]. Our local checker ablation is not differential validation against Azure: the checks observe one implementation at different levels of detail. A live comparison remains necessary for claims that depend on matching Azure behavior. These distinctions position the contribution as a scoped systems and testing artifact, rather than a new general testing theory.

6 Related systems and API drift

Microsoft's Azurite supports local Blob, Queue, and Table development and documents differences from Azure Storage [3]. Microsoft's Service Bus emulator provides local messaging behavior with explicit protocol and persistence limitations [4]. These projects demonstrate the value of a declared support boundary for local development. We have not run a comparative evaluation against them and do not infer relative completeness, performance, or fidelity.

Microsoft's Azure REST API specifications provide a versioned source for reviewing API changes [5]. GlobalStack includes a weekly specification watcher and an opt-in differential harness with selected AKS status and error cases. A changed specification is a review signal, not proof of a changed runtime behavior. Conversely, unchanged documentation does not prove unchanged service behavior. This draft reports no new live differential results.

A future live comparison should run the same versioned scenario against Azure and the local system, retaining request shape, normalized responses, observable side effects, event correlation, and cleanup outcomes. Normalization must be explicit and limited to genuinely variable fields. API version, region, test time, account constraints, and known deviations should accompany every record. Costs and credentials require a separately approved live run. This protocol remains future work for the cross-service scenarios described here.

7 Limitations and release interpretation

Selection and oracle bias. One sample per platform exercises a narrow configuration. The two checker-study flows share the workflow engine. The refined claims path avoids that engine but is also first-party, so neither result is independent external validation. Upstream templates reduce authorship coupling but do not supply live-cloud outcome traces. There is no randomly sampled workload corpus or held-out evaluation. The new baseline comparison is between checking layers on the same local traces; it is not a comparison against another emulator or live Azure.

Local execution model. The tested event paths complete synchronously through loopback HTTP. No retry, timeout distribution, load, network partition, or eventual consistency experiment is reported. The exclusive-outcome check observes only the instrumented receiver within the synchronous invocation interval. It does not exclude effects at uninstrumented services or later times. Hostname adaptation and development identity are harness facilities. The receiver is a test application rather than a deployed Azure compute service.

Security and privacy. The preview is not a production security boundary. Security CI includes dependency auditing, secret scanning, and full-tree JavaScript CodeQL analysis. The previous preview's security correction and remaining reviewed findings are recorded separately in the repository. Passing these checks does not establish absence of vulnerabilities, compliance certification, complete capture sanitization, or suitability for untrusted tenants. Evidence in this draft uses synthetic local identifiers and test values.

Reproduction and provenance. Version pins improve stability but do not freeze the complete software supply chain. Results apply to the tested source revision and retained inputs. Revisions are included in the artifact record for reproducibility, not as evidence of correctness. The refined native case also requires access to a private corpus for full source reproduction; a reviewed source release is pending. The current paper is a technical draft with author information pending and has not been submitted to arXiv.

8 Conclusion

GlobalStack supports a reproducible local study of selected Azure management and service interactions. Nine native reference configurations provide bounded client-execution evidence. A new comparison across 12 configuration mutations separates management acceptance, workflow status, arrival, exact-at-target, and exclusive-outcome checks. Exact-at-target misses two extra-destination mutations despite correct delivery at the intended target; observing the complete receiver interval exposes them. A separate refined Terraform task connects provisioning to observable service behavior and exposes a duplicate Table insert defect that is corrected and rerun. These results support explicitly checking intended outcomes, unwanted effects and service errors in local integration tests. It does not establish comprehensive fault

detection or live Azure equivalence. Independently authored workloads and paired live-cloud measurements are the main next evidence requirements.

A Artifact and reproduction record

Source: https://github.com/divine-ai-com/globalstack. The draft, evidence, and shared-image recipes are in `docs/research/globalstack/`. The historical source preview is `v0.1.0-preview.2`; this draft includes subsequent changes and does not silently replace that tag's results.

Base runtime and native matrix revision: `0f9071b993d3`.

TypeScript dependency rerun revision: `0f523fd77add`.

Flow evidence revision: `0f9071b993d3`.

Full revisions, image identifiers, commands, and limitations are retained in the adjacent JSON records.

Minimum free disk during the native runs: 103.25 GiB.

B Study artifact and claim boundaries

The new protocol is `study/PROTOCOL.md`. The retained `study/evidence/results.json` contains all 36 attempts and 108 episodes, with source revision, clean-tree status, source digests, Node version, and UTC timestamps. The runner is `scripts/e2e/run-flow-study.mjs`; `render-study.mjs` verifies retained traces and regenerates the comparison tables and figure. The native matrix and original eight-episode record retain their earlier provenance. The initial study is retained as `study/evidence/initial-results.json`; the renderer verifies that the corrected measurement record has the same check decisions. The checker study did not rerun native clients or use Azure credentials. The later claims case reruns Terraform separately; its traces, measured contract, known gaps and source digests are retained under `claims/`. The source harness is in the private verification corpus at the recorded revision. The final three runs follow a Table insert correction; pre-fix and development runs are retained separately. `render-claims.mjs` recomputes the final decisions and boundary witnesses. Public source availability for this case remains a release requirement.

Supported claims are execution of the listed native samples, sensitivity differences on the enumerated local faults, observed restoration and cleanup, and three repaired runtime defects. Unsupported claims include service-wide Azure parity, real-world fault prevalence, statistical generalization, superiority to other emulators, production security, and peer-reviewed acceptance. Before publication, the artifact also needs authorship, an independent reproduction or review, and a stable release matching the final manuscript.

References

[1] Microsoft Azure. Azure Quickstart Templates. Pinned fixture revision and file digests are retained in the artifact. https://github.com/Azure/azure-quickstart-templates.

[2] Microsoft. Reference guide to functions in expressions for Azure Logic Apps and Power Automate. https://learn.microsoft.com/en-us/azure/logic-apps/expression-functions-reference. Accessed November 2026.

[3] Microsoft. Use the Azurite emulator for local Azure Storage development. https://learn.microsoft.com/en-us/azure/storage/common/storage-use-azurite. Accessed November 2026.

[4] Microsoft. Overview of the Azure Service Bus emulator. https://learn.microsoft.com/en-us/azure/service-bus-messaging/overview-emulator. Accessed November 2026.

[5] Microsoft Azure. Azure REST API specifications. https://github.com/Azure/azure-rest-api-specs. Accessed November 2026.

[6] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. https://doi.org/10.1109/TSE.2014.2372785.

[7] G. Petrović, M. Ivanković, G. Fraser, and R. Just. Does mutation testing improve testing practices? 2021. https://arxiv.org/abs/2103.07189.

[8] W. M. McKeeman. Differential Testing for Software. *Digital Technical Journal*, 10(1):100–107, 1998. https://www.cs.tufts.edu/comp/150FP/archive/bill-mckeeman/DifferentailTesting.pdf.

[9] Microsoft. Azure Event Grid message delivery and retry. https://learn.microsoft.com/en-us/azure/event-grid/delivery-and-retry. Accessed November 2026.

[10] Microsoft. Endpoint validation with Event Grid event schema. https://learn.microsoft.com/en-us/azure/event-grid/end-point-validation-event-grid-events-schema. Accessed November 2026.

[11] Microsoft. Table Storage error codes. https://learn.microsoft.com/en-us/rest/api/storageservices/table-service-error-codes. Accessed November 2026.